

DOCUMENTACIÓN TÉCNICA PROFUNDA

CitaController - Sistema de Gestión Veterinaria

Módulo: Gestión de Citas

Archivo: app/Http/Controllers/Cita/CitaController.php

Versión: 1.0

Framework: Laravel 12 | PHP 8.2+

Fecha: 15 de Diciembre de 2025

TABLA DE CONTENIDOS

1. Introducción y Propósito
2. Análisis de Dependencias
3. Métodos del Controlador
4. Método index() - Análisis Profundo
5. Método store() - Análisis Profundo
6. Método show() - Análisis Profundo
7. Métodos Incompletos (create, edit, update, destroy)
8. Validación y Seguridad
9. Manejo de Errores
10. Optimizaciones y Mejoras
11. Flujos de Ejecución
12. Testing y Recomendaciones

1. INTRODUCCIÓN Y PROPÓSITO

¿Qué es CitaController?

CitaController es un controlador REST que gestiona todas las operaciones relacionadas con **citas veterinarias** en la clínica. Implementa el patrón CRUD (Create, Read, Update, Delete) aunque actualmente solo tiene implementados parcialmente los métodos.

Responsabilidades Principales

- **Listar Citas:** Obtener todas las citas categorizadas por fecha
- **Crear Citas:** Registrar nuevas citas con validación
- **Obtener Detalles:** Recuperar información completa de una cita
- **Actualizar Citas:** Modificar datos de citas existentes (no implementado)
- **Eliminar Citas:** Cancelar/eliminar citas (no implementado)

Ubicación en la Arquitectura

El controlador se ubica en la capa **Controllers** de la arquitectura MVC:

Arquitectura de Carpetas: app/ ??? Http/ ? ??? Controllers/ ? ??? Cita/ ? ??? CitaController.php ? Está aquí ??? Models/ ? ??? Cita/ ? ? ??? Cita.php ? ??? Mascota/ ? ? ??? Mascota.php ? ??? Profesional.php routes/ ??? citas/ ??? index.php

2. ANÁLISIS DE DEPENDENCIAS

Imports y Namespaces

Import	Propósito	Ubicación
App\Http\Controllers\Controller	Clase base del controlador	app/Http/Controllers/Controller.php
Illuminate\Http\Request	Manejo de solicitudes HTTP	Laravel Framework
Carbon\Carbon	Manipulación de fechas/horas	Librería Carbon
App\Models\Cita\Cita	Modelo de Cita	app/Models/Cita/Cita.php
App\Models\Mascota\Mascota	Modelo de Mascota	app/Models/Mascota/Mascota.php
App\Models\Profesional	Modelo de Profesional	app/Models/Profesional.php

Relaciones Entre Modelos

Flujo de Relaciones:

Cita ? Mascota (belongsTo)

Mascota ? Propietario (belongsTo)

Cita ? Profesional (belongsTo)

Eager Loading:

El controlador carga las relaciones con: `with(['mascota.propietario', 'profesional'])`

Esto evita el problema N+1 en consultas.

3. MÉTODOS DEL CONTROLADOR

Resumen de Métodos

Método	Ruta HTTP	Estado	Propósito
index()	GET /citas	? Completo	Listar todas las citas categorizadas
create()	GET /citas/create	? Vacío	Mostrar formulario de creación
store()	POST /citas	? Completo	Guardar nueva cita en BD
show()	GET /citas/{id}	? Completo	Obtener detalles de una cita
edit()	GET /citas/{id}/edit	? Vacío	Mostrar formulario de edición
update()	PUT /citas/{id}	? Vacío	Actualizar cita existente
destroy()	DELETE /citas/{id}	? Vacío	Eliminar/cancelar cita

4. MÉTODO INDEX() - ANÁLISIS PROFUNDO

Firma del Método

```
public function index() { // ... código }
```

Propósito

Obtener todas las citas del sistema categorizadas por estado temporal (hoy, próximas, pasadas) y devolverlas a la vista de recepción.

Flujo de Ejecución

1. Cargar listados de mascotas y profesionales 2. Definir fecha actual (hoy) 3. Crear query base de citas con eager loading 4. Clonar query para obtener 4 colecciones diferentes: - citasTodas: Todas las citas - citasHoy: Citas del día actual - citasProximas: Citas futuras - citasPasadas: Citas históricas 5. Retornar vista con datos compactados

Desglose de Código

1. Cargar Mascotas

```
$mascotas = Mascota::orderBy('nombre')->get(['id_mascota', 'nombre']);
```

¿Qué hace? Obtiene todas las mascotas ordenadas alfabéticamente

Selección de Columnas: Solo id_mascota y nombre (optimización)

Uso: Llenar dropdown en formulario de citas

2. Cargar Profesionales

```
$profesionales = Profesional::orderBy('nombre')->get(['rfc', 'nombre']);
```

¿Qué hace? Obtiene todos los veterinarios ordenados por nombre

Identificador Único: RFC (Registro Federal de Contribuyentes)

Uso: Llenar dropdown de profesionales disponibles

3. Definir Fecha Actual

```
$hoy = Carbon::today();
```

¿Qué es Carbon? Librería para manejo de fechas en Laravel

Carbon::today(): Retorna la fecha actual sin hora (00:00:00)

Ventaja: Comparaciones de fecha exacta sin problemas de hora

4. Query Base con Eager Loading

```
$citasQuery = Cita::with(['mascota.propietario', 'profesional']) ->orderBy('fecha') ->orderBy('horario');
```

with(): Eager loading de relaciones

mascota.propietario: Carga en cascada

Problema N+1 Evitado: Sin esto harían múltiples queries

Ordenamiento: Por fecha y luego por hora

5. Clonación de Query

```
$citasTodas = (clone $citasQuery)->get(); $citasHoy = (clone $citasQuery)->whereDate('fecha', $hoy)->get(); $citasProximas = (clone $citasQuery)->whereDate('fecha', '>', $hoy)->get(); $citasPasadas = (clone $citasQuery)->whereDate('fecha', '<', $hoy)->get();
```

¿Por qué clone? Evitar que los where() afecten la query original

whereDate(): Compara solo la fecha, ignora la hora

Resultado: 4 colecciones independientes con citas categorizadas

Retorno del Método

Tipo: Vista Blade (view)

Ubicación: resources/views/dash/recepcion/citas.blade.php

Parámetros Pasados:

- mascotas (Collection)
- profesionales (Collection)
- citasHoy (Collection)
- citasProximas (Collection)
- citasPasadas (Collection)
- citasTodas (Collection)

Complejidad y Performance

Aspecto	Análisis
Queries SQL	3 queries principales (Mascotas, Profesionales, Citas con relaciones)

Eager Loading	? Optimizado (evita N+1)
Índices Recomendados	fecha, profesional_id, mascota_id en tabla citas
Escalabilidad	Con 10K+ citas podría necesitar paginación

5. MÉTODO STORE() - ANÁLISIS PROFUNDO

Firma del Método

```
public function store(Request $request) { // ... validación y creación }
```

Propósito

Validar datos de solicitud HTTP y crear una nueva cita en la base de datos.

Flujo de Ejecución

```
1. Validar datos de entrada con reglas específicas 2. Si validación falla? Retornar errores (Laravel automático) 3. Parsear fecha a formato Y-m-d 4. Crear registro en tabla citas 5. Retornar respuesta JSON con código 201
```

Validación de Datos

Campo	Reglas	Descripción
id_mascota	required exists:mascota,id_mascota	Debe existir mascota en BD con ese ID
rfc_profesional	required exists:profesional,rfc	Debe existir profesional con ese RFC
tipo_servicio	nullable string max:100	Opcional, máximo 100 caracteres
tipo_cita	required in:Consulta,Urgencia,Cirugía,Estética	Solo valores específicos permitidos
tarifa	nullable numeric min:0	Opcional, número positivo
peso_mascota	nullable numeric min:0	Opcional, peso en kg sin negativos

fecha	required date	Fecha válida, formato YYYY-MM-DD
horario	required date_format:H:i	Hora en formato 24h, ej: 14:30
diagnostico	nullable string	Diagnóstico veterinario (opcional)
observaciones	nullable string	Notas adicionales (opcional)
estado	required in:Programada,Completada,Cancelada	Estado actual de la cita

Transformación de Datos

```
$data['fecha'] = Carbon::parse($data['fecha'])->format('Y-m-d');
```

¿Por qué? Asegurar formato consistente en BD

Carbon::parse(): Interpreta varios formatos de fecha

format(): Estandariza a YYYY-MM-DD

Ejemplo: "15/12/2025" ? "2025-12-15"

Creación de Registro

```
$cita = Cita::create($data);
```

Modelo Mass Assignment: Cita.php debe permitir estos campos

Retorno: Instancia del modelo Cita creado

Evento Disparado: created (observable del modelo)

Respuesta JSON

```
return response()->json([ 'success' => true, 'message' => 'Cita creada correctamente', 'data' => $cita ], 201);
```

Código HTTP 201: Recurso creado exitosamente

Estructura JSON:

```
{
  "success": true,
  "message": "Cita creada correctamente",
```

```
"data": { cita_object }  
}
```

Cliente puede usar: `Response.status === 201` para detectar creación

Potenciales Problemas

- **Sin validación de conflictos:** No verifica si profesional ya tiene cita en ese horario
- **Sin validación de horario:** No comprueba que la hora esté dentro de horario laboral
- **Sin validación de capacidad:** No limita citas por profesional por día
- **Fechas pasadas:** Permite crear citas con fecha retroactiva

6. MÉTODO SHOW() - ANÁLISIS PROFUNDO

Firma del Método

```
public function show(string $id) { // ... búsqueda y retorno }
```

Propósito

Obtener detalles completos de una cita específica incluyendo mascota, propietario y profesional.

Estructura de Ejecución

```
1. Try: Intentar obtener la cita 2. Buscar cita por ID con eager loading 3. If: Cita no existe? Retornar 404 4. Else: Retornar JSON con datos 5. Catch: Si hay excepción ? Retornar error 500
```

Búsqueda de Cita

```
$cita = Cita::with(['mascota.propietario', 'profesional'])->find($id);
```

find(\$id): Busca por primary key (id)

with(): Eager loading de relaciones

Retorno: Objeto Cita o null

Validación de Existencia

```
if (!$cita) { return response()->json([ 'success' => false, 'message' => 'Cita no encontrada' ], 404); }
```

Código 404: Recurso no encontrado

Respuesta JSON: Estructura consistente

¿Qué hacer? Cliente puede redirigir a 404 o mostrar mensaje

Respuesta Exitosa

```
return response()->json([ 'success' => true, 'data' => $cita, 'message' => 'Cita obtenida correctamente' ]);
```

Código HTTP: 200 (implícito)

Estructura: success, data, message

Datos retornados: Cita completa con relaciones cargadas

Manejo de Excepciones

```
catch (\Exception $e) { return response()->json([ 'success' => false, 'message' => 'Error al obtener la cita: ' . $e->getMessage() ], 500); }
```

Código 500: Error interno del servidor

¿Cuándo? Problemas de BD, conexión, etc.

Mensaje: Incluye detalles del error (en producción, cambiar)

Seguridad: No exponer mensajes de error en producción

Datos Retornados (Estructura JSON)

```
{ "success": true, "data": { "id": 1, "id_mascota": 5, "rfc_profesional": "VET123456XYZ", "tipo_servicio": "Consulta General", "tipo_cita": "Consulta", "tarifa": 500.00, "peso_mascota": 15.5, "fecha": "2025-12-15", "horario": "14:30", "diagnostico": "Infección urinaria", "observaciones": "Prescribir antibióticos", "estado": "Completada", "created_at": "2025-12-15T10:30:00Z", "updated_at": "2025-12-15T16:45:00Z", "mascota": { "id_mascota": 5, "nombre": "Firulais", "propietario": { ... } }, "profesional": { "rfc": "VET123456XYZ", "nombre": "Dr. García" } }, "message": "Cita obtenida correctamente" }
```

7. MÉTODOS INCOMPLETOS (create, edit, update, destroy)

Estado Actual

Los siguientes métodos están vacíos y necesitan implementación:

create()

```
public function create(Request $request) { // }
```

Propósito: Retornar vista con formulario para crear nueva cita

Implementación sugerida:

```
public function create() { $mascotas = Mascota::orderBy('nombre')->get(); $profesionales = Profesional::orderBy('nombre')->get(); return view('dash.recepcion.citas.create', compact('mascotas', 'profesionales')); }
```

edit()

```
public function edit(string $id) { // }
```

Propósito: Retornar vista con formulario para editar cita existente

Implementación sugerida:

```
public function edit(string $id) { $cita = Cita::findOrFail($id); $mascotas = Mascota::orderBy('nombre')->get(); $profesionales = Profesional::orderBy('nombre')->get(); return view('dash.recepcion.citas.edit', compact('cita', 'mascotas', 'profesionales')); }
```

update()

```
public function update(Request $request, string $id) { // }
```

Propósito: Actualizar cita existente con validación

Implementación sugerida:

```
public function update(Request $request, string $id) { $cita = Cita::findOrFail($id); $data = $request->validate([ 'id_mascota' => 'sometimes|required|exists:mascota,id_mascota', 'rfc_profesional' => 'sometimes|required|exists:profesional,rfc', 'tipo_cita' => 'sometimes|required|in:Consulta,Urgencia,Cirugia,Estetica', 'estado' => 'sometimes|required|in:Programada,Completada,Cancelada', // ... otros campos ]); if(isset($data['fecha'])) { $data['fecha'] = Carbon::parse($data['fecha'])->format('Y-m-d'); } $cita->update($data); return response()->json([ 'success' => true, 'message' => 'Cita actualizada correctamente', 'data' => $cita ]); }
```

destroy()

```
public function destroy(string $id) { // }
```

Propósito: Cancelar/eliminar cita

Implementación sugerida:

```
public function destroy(string $id) { $cita = Cita::findOrFail($id); // Opción 1: Soft delete (borrado lógico) $cita->delete(); // 0  
Opción 2: Cambiar estado (mejor para auditoría) // $cita->update(['estado' => 'Cancelada']); return response()->json([ 'success' => true, 'message' => 'Cita cancelada correctamente' ]); }
```

8. VALIDACIÓN Y SEGURIDAD

Validación de Entrada

Laravel valida automáticamente los datos en el método `store()`:

- **Exists Validator:** Comprueba referential integrity
- **In Validator:** Restringe valores a opciones válidas
- **Date Validator:** Asegura formato de fecha correcto
- **Date Format:** Valida exactamente H:i

Protecciones Implementadas

CSRF Protection: Laravel protege contra ataques CSRF automáticamente

SQL Injection: Eloquent ORM usa prepared statements

Type Hinting: `string $id` asegura tipo correcto

Exception Handling: Try-catch en `show()` captura errores

Vulnerabilidades Detectadas

- **Sin autenticación:** No verifica que usuario esté logueado
- **Sin autorización:** No comprueba permisos (rol admin/médico/recepción)
- **Sin rate limiting:** No limita cantidad de requests
- **Sin auditoría:** No registra quién hizo cambios
- **Exposición de errores:** En `show()` expone mensajes de excepción

Mejoras Recomendadas

```
// Agregar en constructor public function __construct() { $this->middleware('auth'); $this->middleware('can:view-citas'); } // O usar  
gate/policy en cada método public function show(string $id) { $cita = Cita::findOrFail($id); $this->authorize('view', $cita); // ... }
```


9. MANEJO DE ERRORES

Errores en Método store()

Escenario	Respuesta	Código
Validación fallida	422 Unprocessable Entity + errores	422
Mascota no existe	Validación falla (exists rule)	422
Profesional no existe	Validación falla (exists rule)	422
Creación exitosa	JSON con datos + código	201

Errores en Método show()

Escenario	Respuesta	Código
Cita no existe	JSON con error	404
Error BD/Conexión	JSON con excepción	500
Cita encontrada	JSON con datos completos	200

Ejemplo de Error JSON en Producción

```
{ "success": false, "message": "Cita no encontrada", "errors": {} }
```

IMPORTANTE: En archivo config/app.php, APP_DEBUG debe ser false en producción para no exponer detalles internos.

10. OPTIMIZACIONES Y MEJORAS

Problema N+1 - Ya Optimizado ?

```
// MAL: Causa múltiples queries foreach($citas as $cita) { echo $cita->mascota->nombre; // Query por cada cita echo $cita->profesional->nombre; // Otra query por cita } // ? BIEN: Una sola query (implementado en controlador) $citas = Cita::with(['mascota', 'profesional'])->get();
```

Paginación Recomendada

```
// Modificar index() para paginar public function index() { $citasHoy = Cita::whereDate('fecha', today()) ->with(['mascota.propietario', 'profesional']) ->paginate(15); return view('dash.recepcion.citas', compact('citasHoy')); }
```

Caché para Listados Estáticos

```
// Mascotas y profesionales cambian raramente public function index() { $mascotas = Cache::remember('mascotas', 60, function() { return Mascota::orderBy('nombre')->get(['id_mascota', 'nombre']); }); $profesionales = Cache::remember('profesionales', 60, function() { return Profesional::orderBy('nombre')->get(['rfc', 'nombre']); }); // ... }
```

Validación de Horarios Disponibles

```
// Antes de crear cita, verificar disponibilidad $conflicto = Cita::where('rfc_profesional', $data['rfc_profesional']) ->where('fecha', $data['fecha']) ->where('horario', $data['horario']) ->where('estado', '!=', 'Cancelada') ->exists(); if($conflicto) { return response()->json([ 'success' => false, 'message' => 'Horario no disponible' ], 422); }
```

Índices de Base de Datos

```
// Agregar a migración de citas Schema::create('cita', function (Blueprint $table) { // ... campos $table->index('fecha'); $table->index('rfc_profesional'); $table->index('id_mascota'); $table->index(['rfc_profesional', 'fecha']); // Índice compuesto });
```

11. FLUJOS DE EJECUCIÓN

Flujo 1: Listar Citas (GET /citas)

Usuario abre página de citas ? GET /citas dispara CitaController@index() ? index() carga mascotas y profesionales ? index() carga citas categorizadas por fecha ? Retorna vista con datos compactados ? Blade renderiza HTML con tablas de citas ? Usuario ve: Citas hoy, próximas, pasadas

Flujo 2: Crear Cita (POST /citas)

Usuario completa formulario y presiona "Guardar" ? JavaScript (AJAX) envia POST /citas con datos ? CitaController@store() recibe Request ? validate() verifica reglas ? ?? SI falla: Retorna 422 con errores ? ?? SI pasa: Parsea fecha, crea registro ? Cita::create(\$data) inserta en BD ? Retorna JSON 201 con cita creada ? JavaScript recibe respuesta ? ??? Mostrar succes message ??? Limpiar formulario ??? Recargar tabla de citas

Flujo 3: Ver Detalles (GET /citas/{id})

Usuario hace clic en una cita ? GET /citas/5 dispara CitaController@show(5) ? show() busca Cita con ID 5 ? ?? NO existe: Retorna JSON 404 ? ?? Existe: Carga relaciones (mascota, profesional) ? Retorna JSON 200 con datos completos ? JavaScript recibe datos ? Muestra modal/página con detalles

12. TESTING Y RECOMENDACIONES

Tests Unitarios Recomendados (Pest)

```
// tests/Feature/CitaControllerTest.php use Tests\TestCase; class CitaControllerTest extends TestCase { /** @test */ public function puede_listar_citas() { $response = $this->get('/citas'); $response->assertStatus(200); } /** @test */ public function puede_crear_cita() { $data = [ 'id_mascota' => 1, 'rfc_profesional' => 'VET123', 'tipo_cita' => 'Consulta', 'fecha' => '2025-12-20', 'horario' => '14:00', 'estado' => 'Programada' ]; $response = $this->post('/citas', $data); $response->assertStatus(201); $this->assertDatabaseHas('citas', $data); } /** @test */ public function puede_obtener_cita() { $cita = Cita::factory()->create(); $response = $this->get("/citas/{$cita->id}"); $response->assertStatus(200) ->assertJson(['success' => true]); } }
```

Recomendaciones Finales

- ? Completar métodos create(), edit(), update(), destroy()
- ? Agregar autenticación y autorización
- ? Implementar validación de horarios disponibles
- ? Agregar paginación para grandes volúmenes
- ? Implementar caché para datos estáticos
- ? Agregar logging/auditoría de cambios
- ? Escribir tests unitarios e integración
- ? Documentar API con Swagger/OpenAPI
- ? Configurar rate limiting
- ? Usar Form Requests para validación reutilizable

